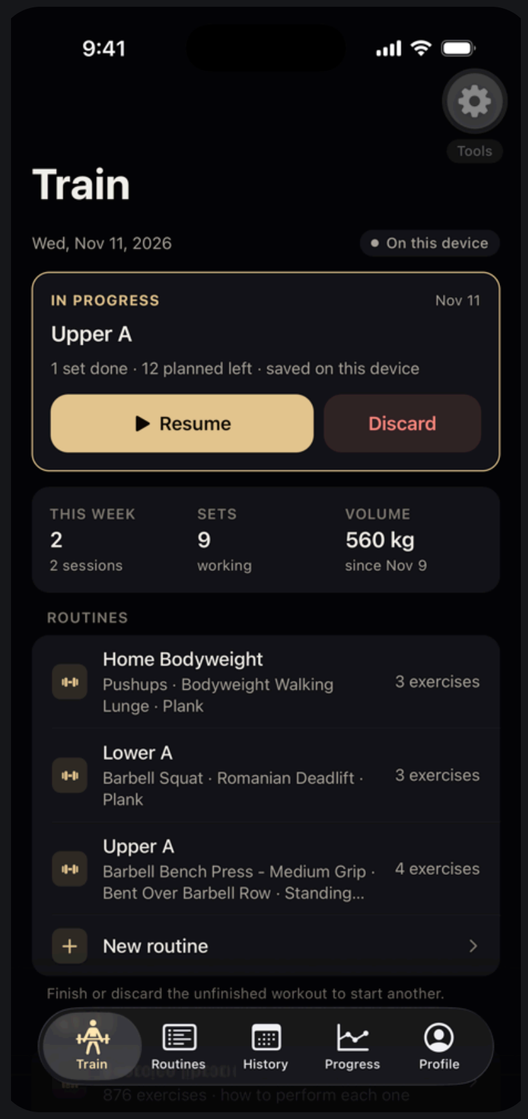
CS3365 SOFTWARE ENGINEERING · PROJECT 2

From diagrams to a one-shot app

Fitness & Workout Log: the design artifacts, how we iterated them, and a live demo of two generated apps, one by OpenAI Codex and one by Claude.

Sebastian · Diego · November 12, 2026



OUR SELECTION · OPTION 1 OF 4

We chose the Claude Fitness & Workout Log

People who train regularly log sets in notes, paper or spreadsheets. Names drift, last week’s numbers are hard to find at the rack, and nobody can see whether they are actually improving.

31
user stories, FW-01 to FW-31

101
story points, 9 one-pagers

After the professor’s review, personas are named by role only, and every story names a role.

Experienced lifter
Trains five days a week. Needs fast entry between sets, consistent names and volume graphs.

Beginner lifter
Three months in. Needs last time’s weights, a simple flow and a plain summary.

Home trainer
Shift worker at home. Needs reps-only and timed sets, and logging with no signal.

Returning lifter
Low tech comfort. Needs large text, kg and lb at two gyms, a readable history.

Source: Fitness_Workout_Log_App_Revised_v3.docx (professor-reviewed 1-pagers) · 2

WHAT WE TOOK FROM THE 1-PAGERS

Six needs that shaped the design

Users need…	So the app…
Some exercises have no weight	only asks for what applies: weight, reps or seconds
Never lose a set, even offline	saves on the phone first and uploads later, never twice
Exercise names stay consistent	gives each exercise a fixed ID, so renaming keeps its history
Fix mistakes after a workout	updates history and graphs everywhere after an edit or delete
Some gyms use kg, others lb	saves the weight as typed and converts only on screen
Text that is easy to read	lets text grow to 200%, and every chart also lists its numbers

We also added planned training: routines like 4 sets of 12, a rest timer on the Lock Screen, warm-ups, 876 exercises, a muscle map, reports and data export.

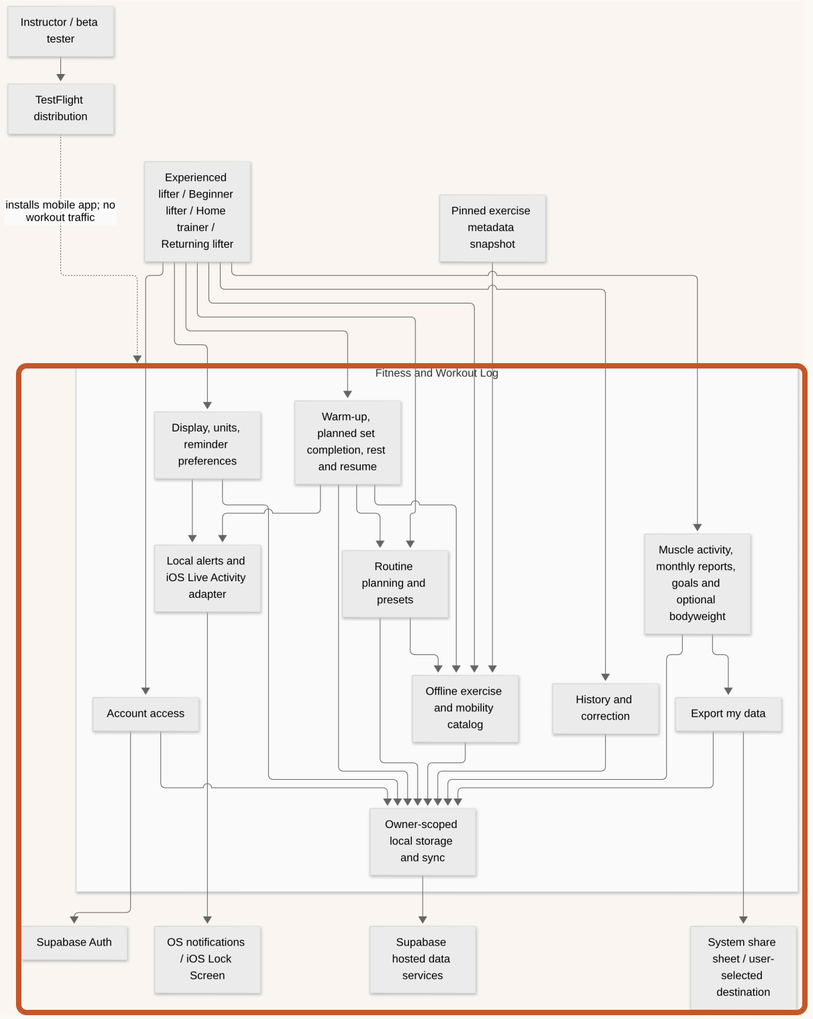
Source: the nine 1-pagers in Fitness_Workout_Log_App_Revised_v3.docx · 3

THE DESIGN

50 Mermaid diagrams, each reviewed in its own pull request

1	1	1	2	5	40
Context model	Architecture	Navigation	Data models	Class sheets	Sequences

- 40 sequences = 31 story flows (one per FW story) + 2 supporting flows (sign-in, conflict review) + 7 planned-training flows
- TRACEABILITY.md links every story to its diagram and acceptance check
- check_design.py validates coverage and links; all 50 diagrams render
- Design PRs #2–#45: one diagram per PR, merged in dependency order



CONTEXT MODEL · 01-context.mmd

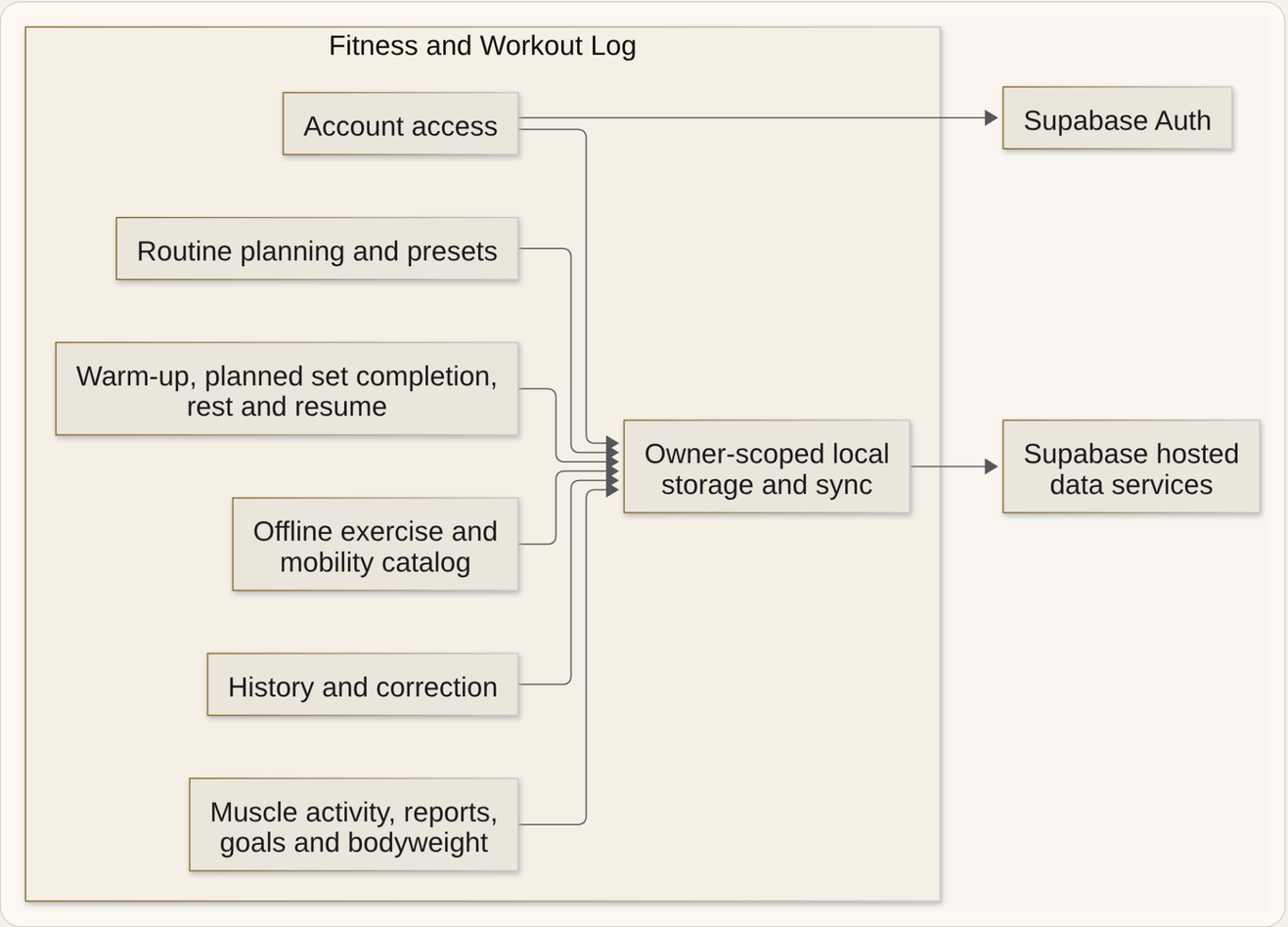
What we build, and what we rely on

Eight modules inside the app, used by four lifter roles. Outside it are Supabase Auth and data, OS notifications and the Lock Screen, the share sheet, a pinned exercise catalog and TestFlight.

- Draws the product boundary early
- Every module writes through one owner-scoped storage and sync module
- Its modules become the packages of the class diagram

CONTEXT MODEL · ZOOM

Every module shares one data path



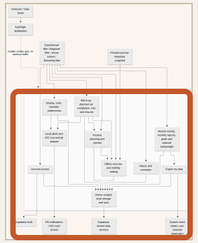
No direct network calls. Planning, logging, catalog, history, profile and account modules never talk to the internet themselves.

One shared module. They read and write through owner-scoped local storage and sync: every record belongs to one signed-in lifter.

Two exits. Only that module talks to Supabase data, and only account access talks to Supabase Auth.

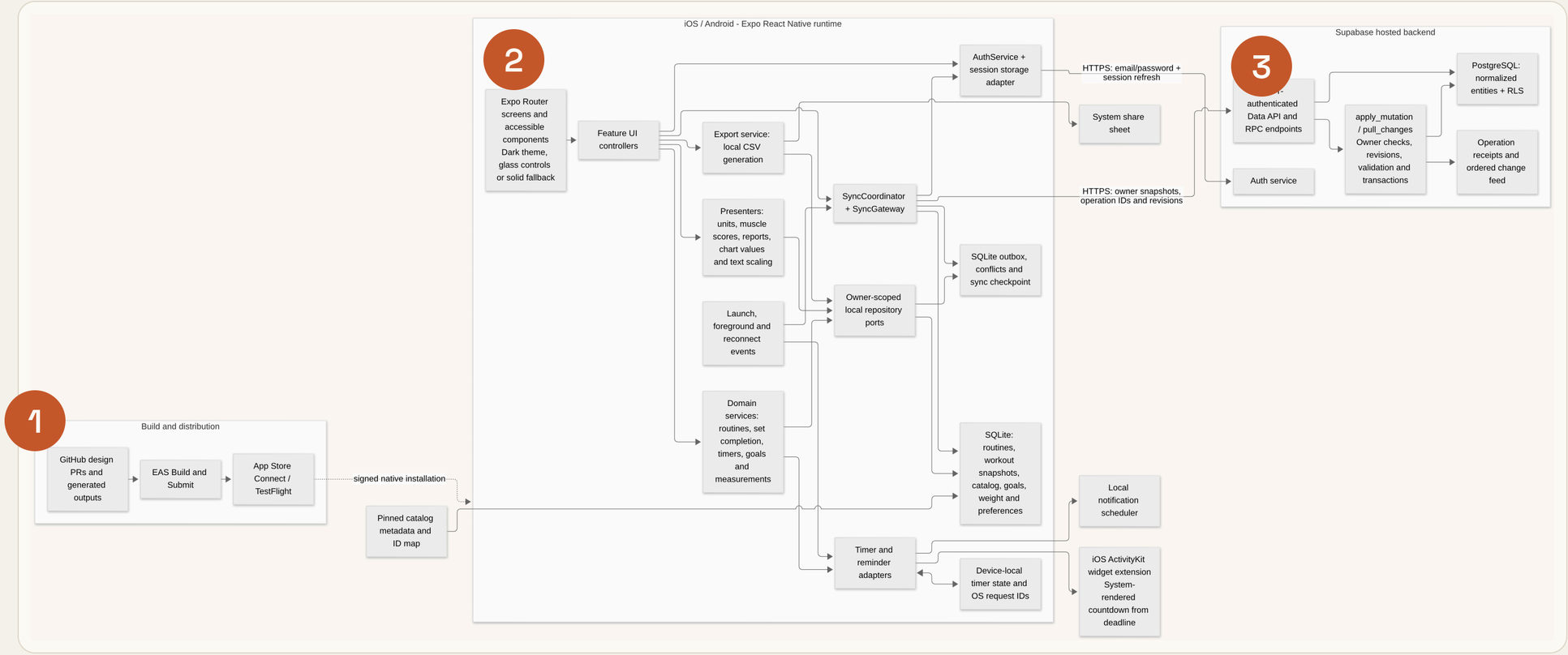
Why it matters: offline durability and safe sync are solved once, in one place.

In the full diagram



ARCHITECTURE · OVERVIEW

Three runtime areas: build, phone, backend



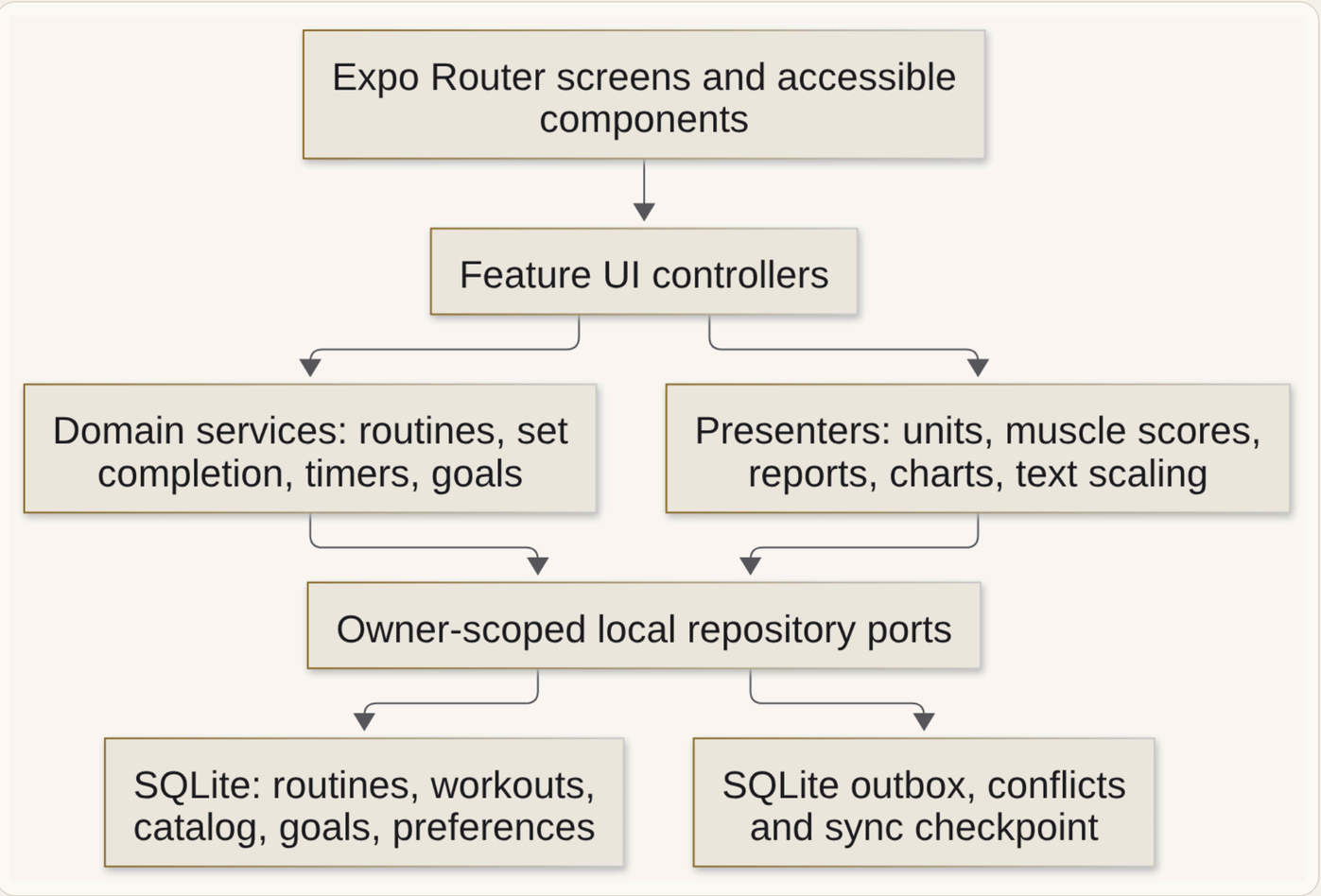
1 Build: GitHub → EAS → TestFlight

2 Phone: screens, services, SQLite

3 Supabase: Auth, RPC, PostgreSQL

ARCHITECTURE · ZOOM 1

Layers inside the phone



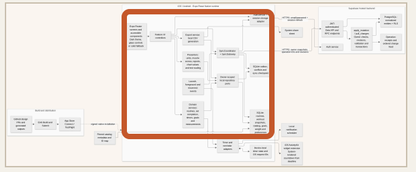
Screens only render; controllers turn taps into calls.

Domain services hold the rules: routines, set completion, timers, goals.

Presenters turn records into what you see: kg or lb, muscle scores, charts, 200% text.

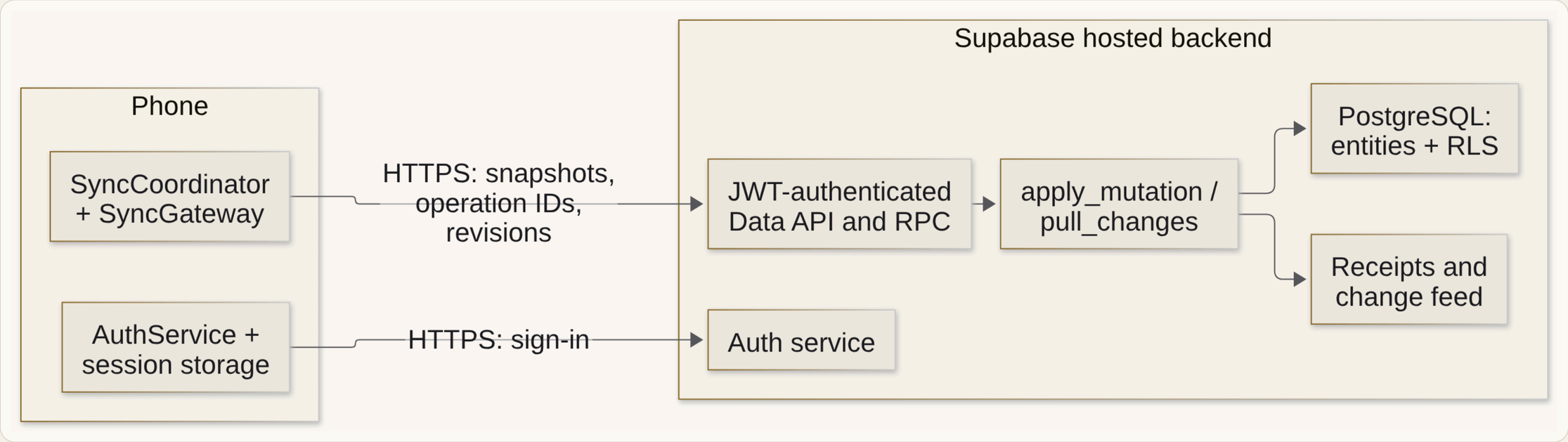
Repositories are owner-scoped ports over SQLite. Each confirmed write goes into SQLite and the outbox in one transaction.

In the full diagram



ARCHITECTURE · ZOOM 2

From the phone to Supabase

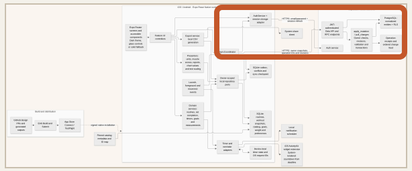


When: sync runs on launch, foreground and reconnect, never while you are logging a set.

How: two narrow RPCs, apply_mutation and pull_changes, check owner, revision and receipt.

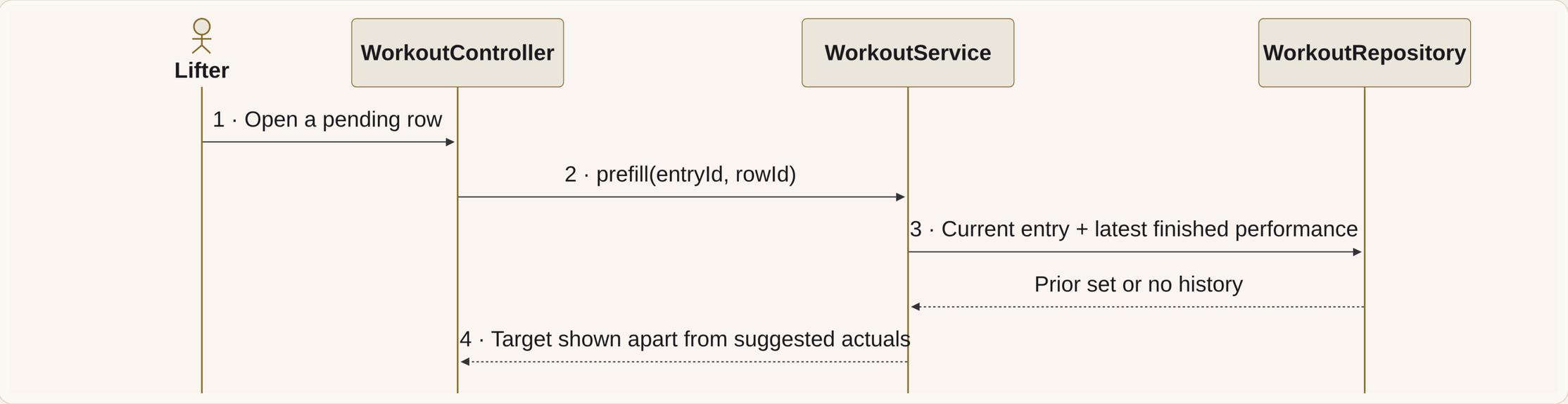
Privacy: PostgreSQL row-level security keeps each lifter’s data private.

In the full diagram



SEQUENCE · FW-02 · ZOOM 1

Record a set: prefill first

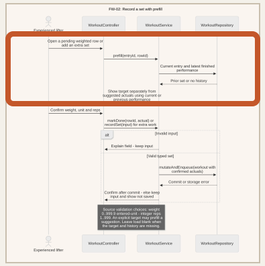


Story: as an experienced lifter, I want my set prefilled from my previous set, so I can log without losing rest time.

Steps 1–3: the controller asks the service, which asks the repository for the latest finished performance.

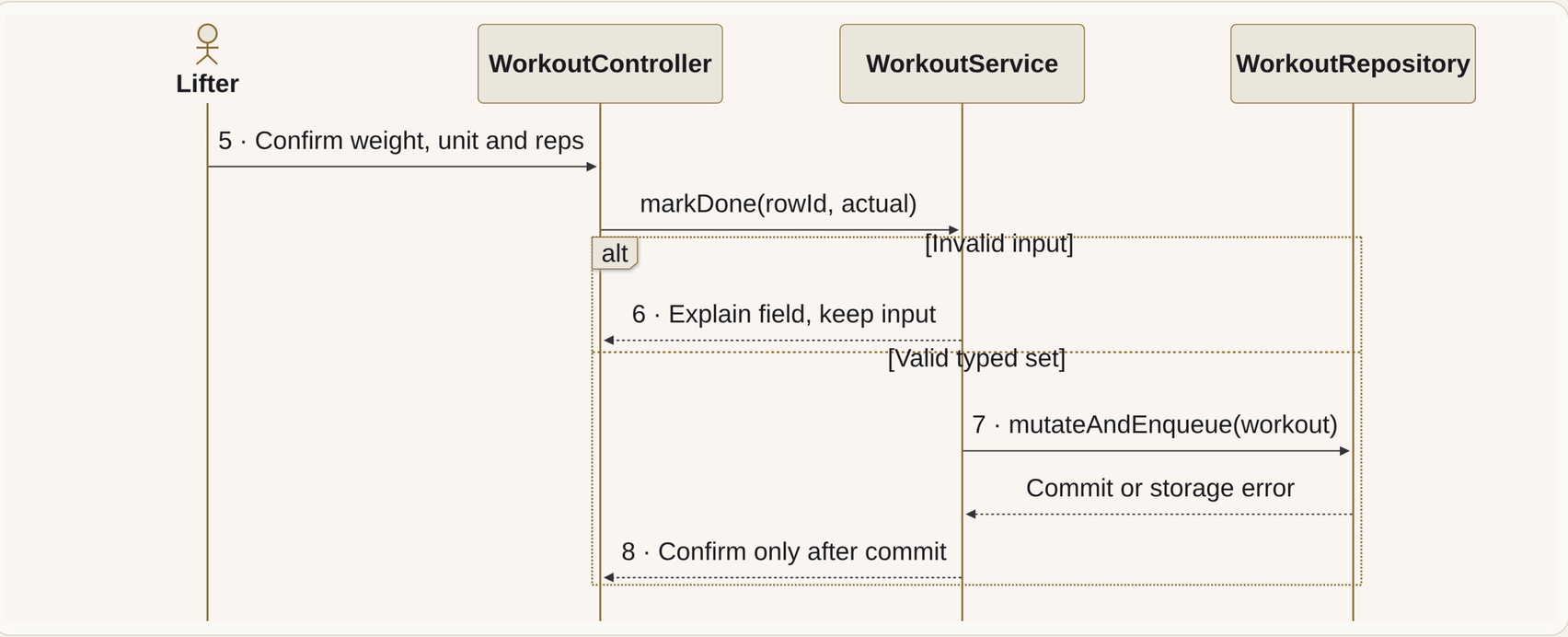
Step 4: the target (say 12 reps) is shown apart from the suggestion, so the plan never changes.

In the full diagram

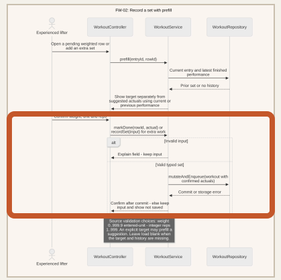


SEQUENCE · FW-02 · ZOOM 2

Record a set: validate, then commit



In the full diagram



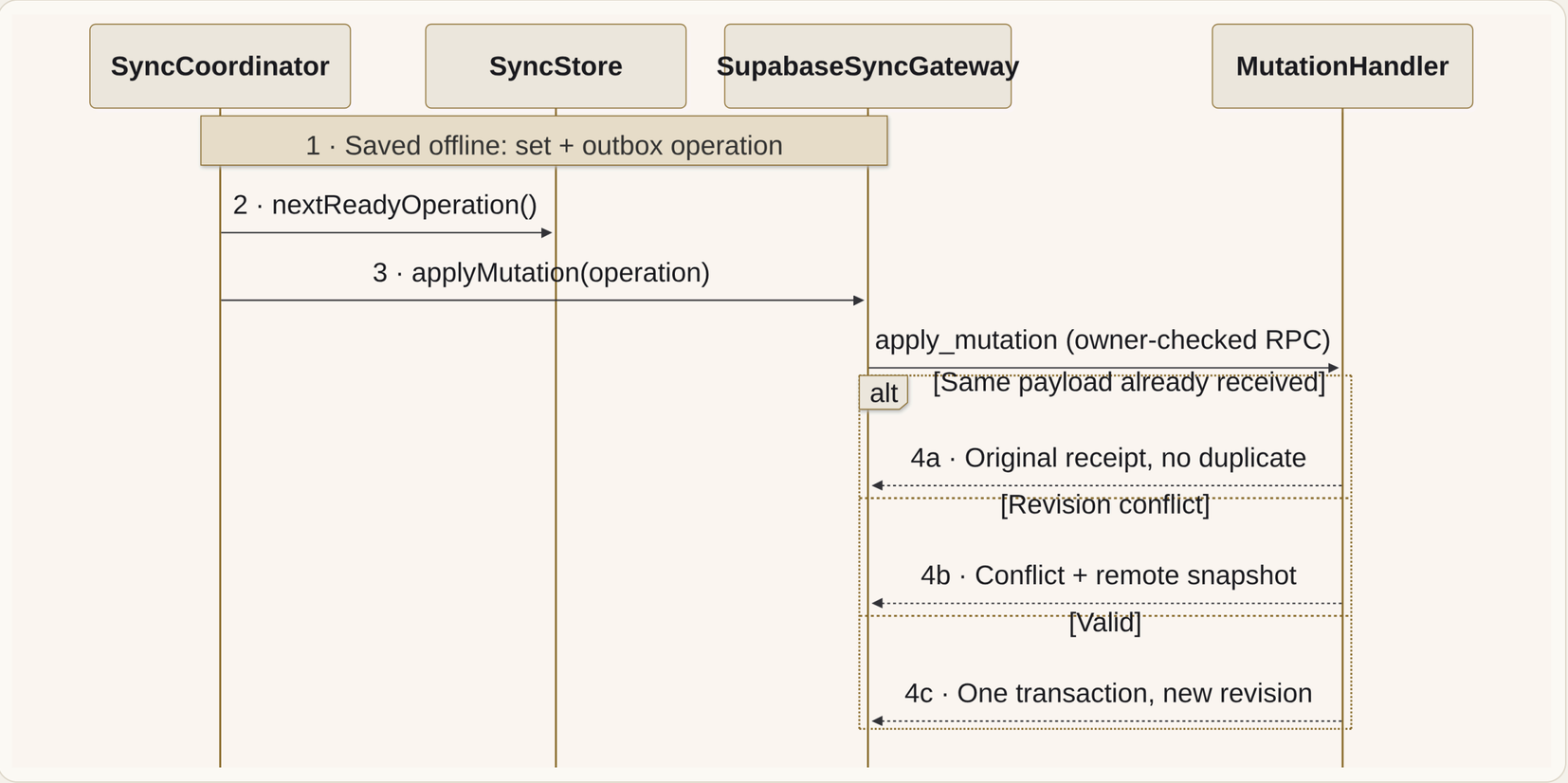
6 · Invalid: say which field is wrong and keep what was typed.

7 · Valid: mutateAndEnqueue saves the set and its upload in one SQLite transaction.

8 · Saved: the set shows as saved only after the commit, so there is no fake success.

SEQUENCE · FW-09 · ZOOM 1

Log offline: upload without duplicates



1 Offline, the set and an outbox operation are saved together.

2–3 On reconnect, the coordinator sends the next operation.

4a Same operation again? The server returns the original receipt.

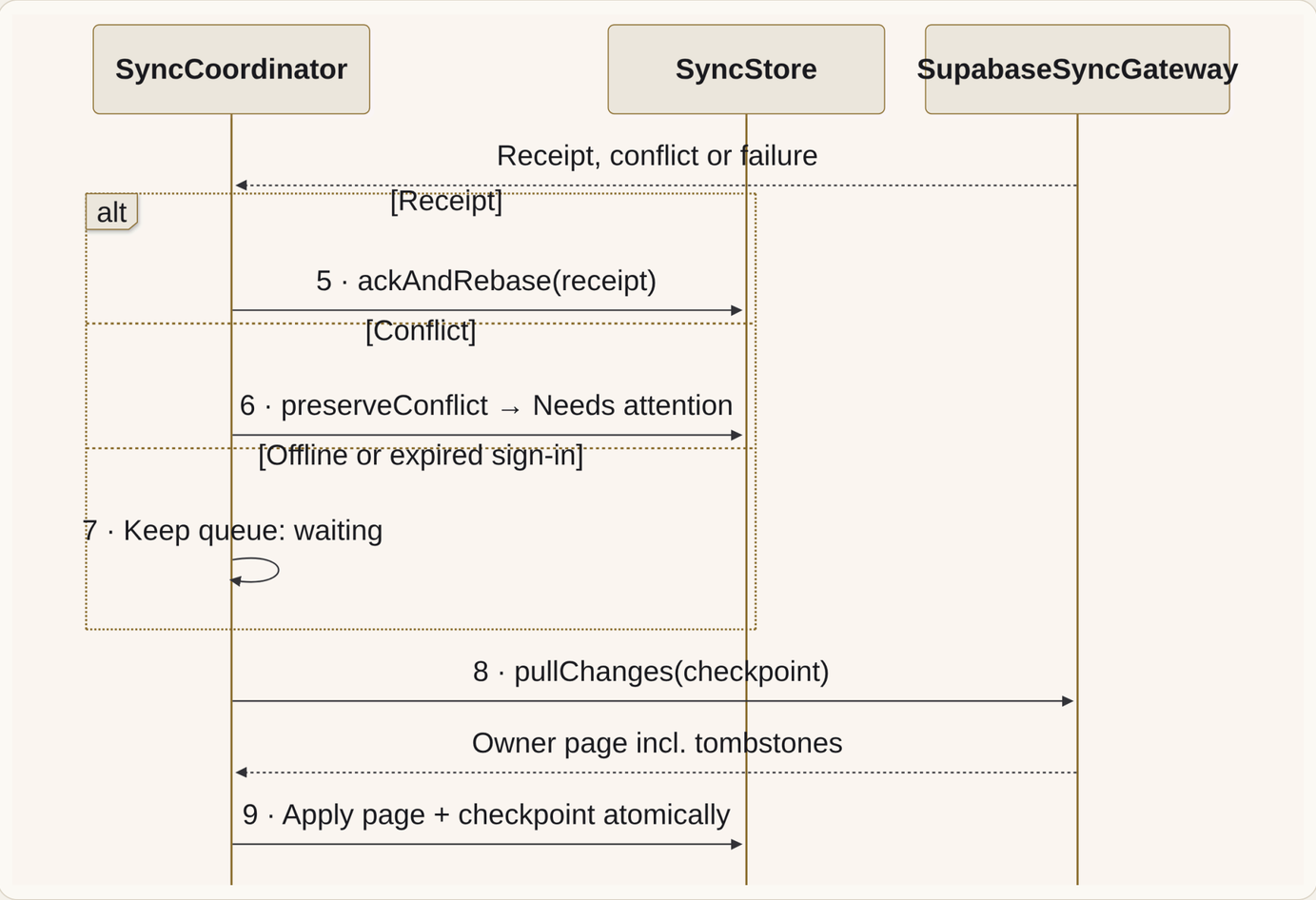
4b · 4c A conflict keeps both versions; a valid change is one transaction.

In the full diagram



SEQUENCE · FW-09 · ZOOM 2

Apply the result, then pull changes



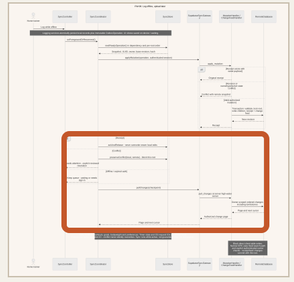
5 · Receipt: mark it uploaded and keep any newer local edits.

6 · Conflict: block that record and show Needs attention.

7 · Offline: keep the queue and retry later.

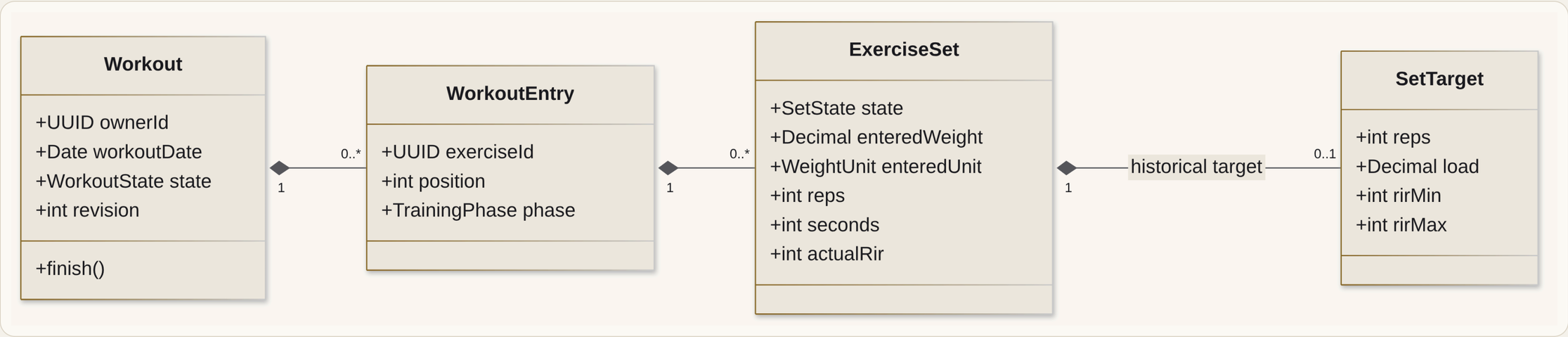
8–9 · Pull: fetch the server's changes, deletions included, and save them with the checkpoint in one step.

In the full diagram



CLASS DIAGRAM · DOMAIN SHEET · ZOOM

The workout aggregate



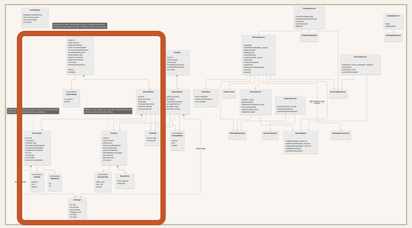
Composition: Workout ▸ WorkoutEntry ▸ ExerciseSet, so a set never exists without its workout.

Units as typed: enteredWeight and enteredUnit are stored; kg and lb conversion happens only on screen.

Plan vs. actual: SetTarget is a snapshot of the plan, so 4 × 12 stays 4 × 12.

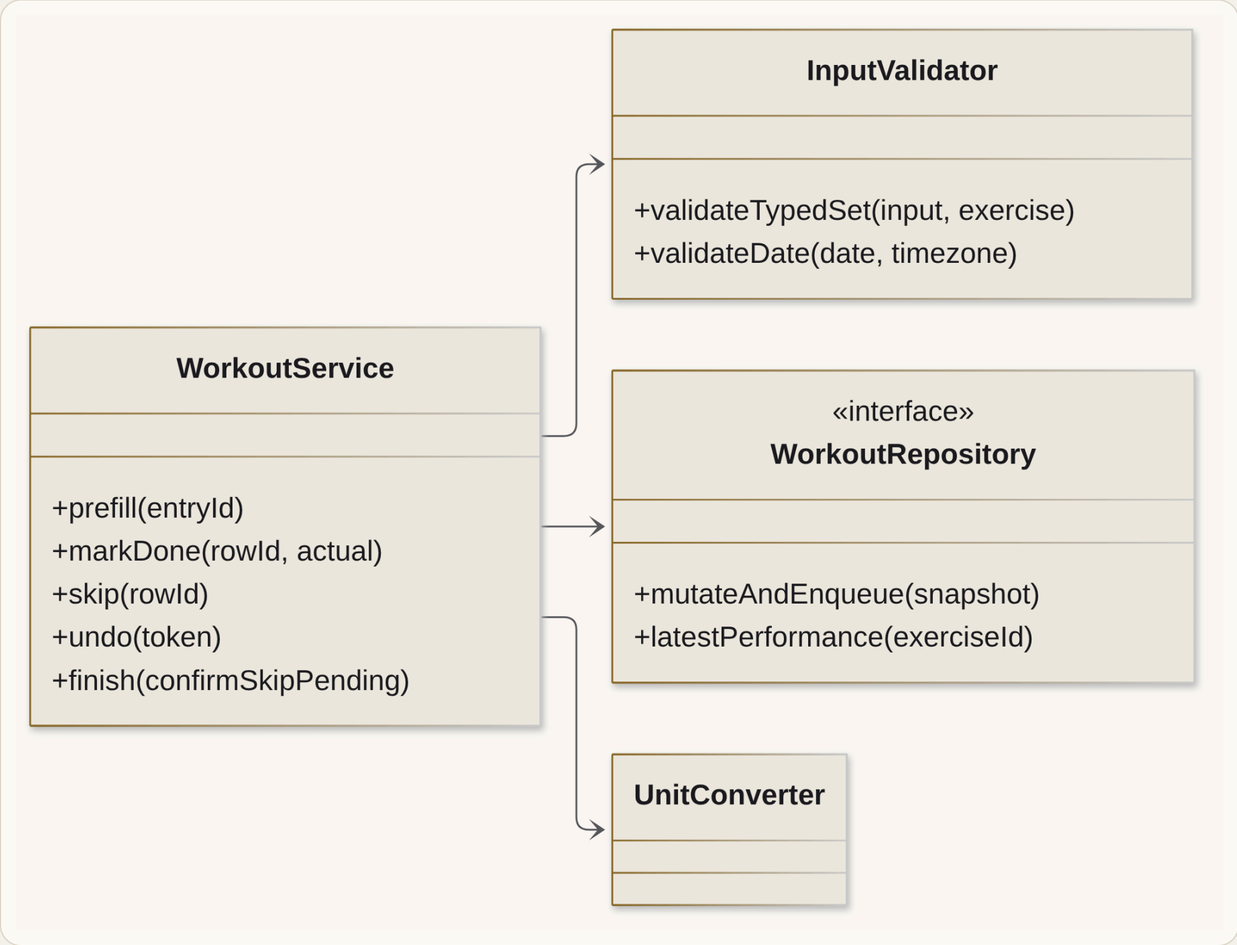
Sync and privacy: ownerId and revision on Workout drive both.

In the full diagram



CLASS DIAGRAM · DOMAIN SHEET · ZOOM

Services the sequence diagrams call



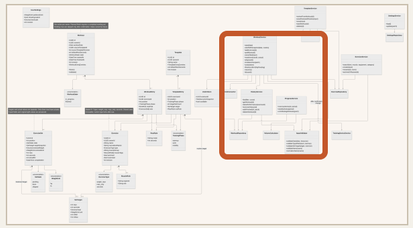
WorkoutService owns the workout lifecycle: prefill, markDone, skip, undo, finish.

InputValidator holds every rule once: typed sets and dates.

WorkoutRepository is an interface, so services never depend on SQLite directly.

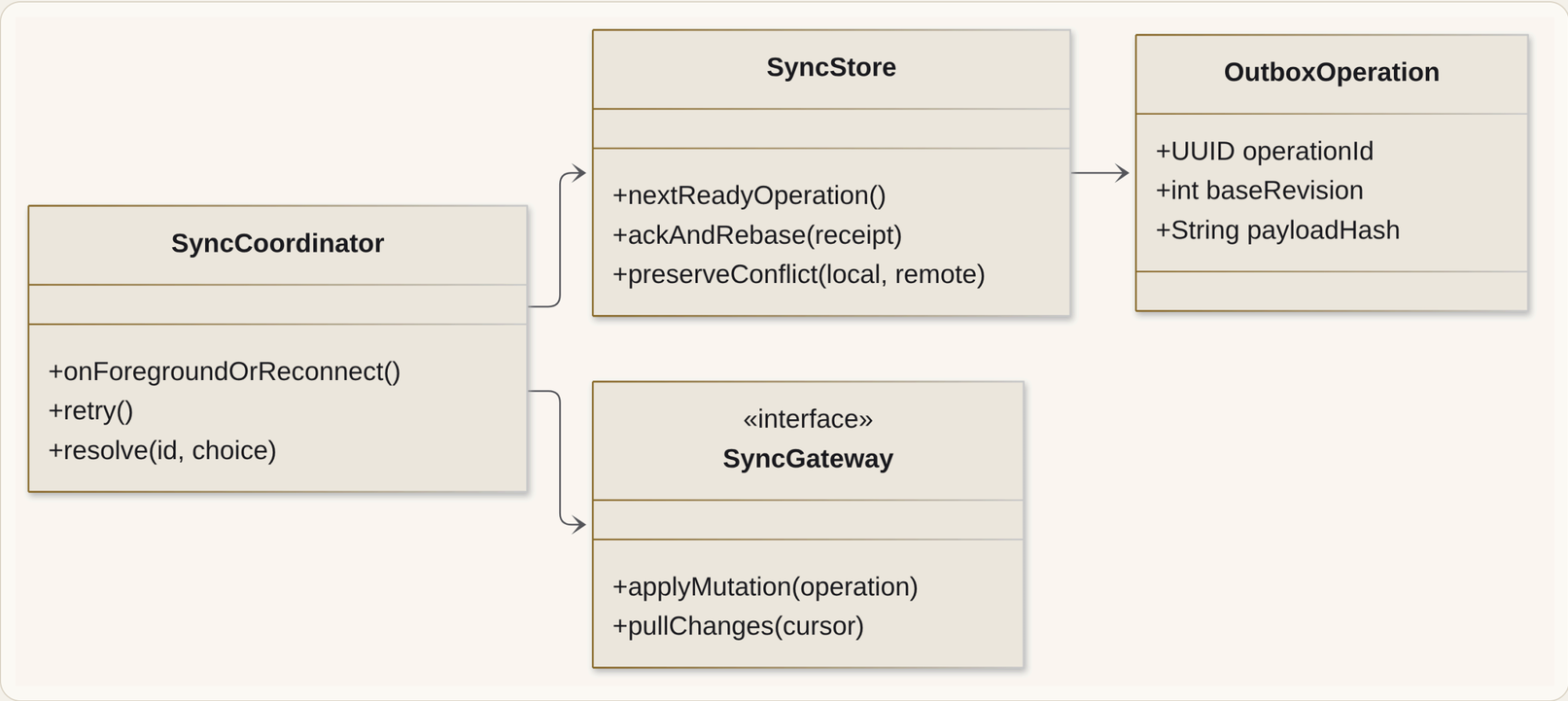
Same calls as the FW-02 sequence: diagrams and classes agree.

In the full diagram



CLASS DIAGRAM · PERSISTENCE AND SYNC · ZOOM 1

Sync classes on the phone



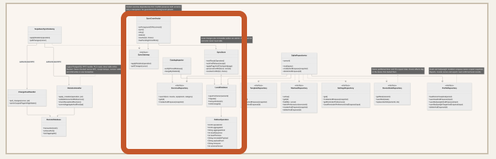
SyncCoordinator decides when to sync and resolves conflicts.

SyncStore reads the outbox and records each result.

OutboxOperation: operationId, baseRevision and payloadHash make retries safe.

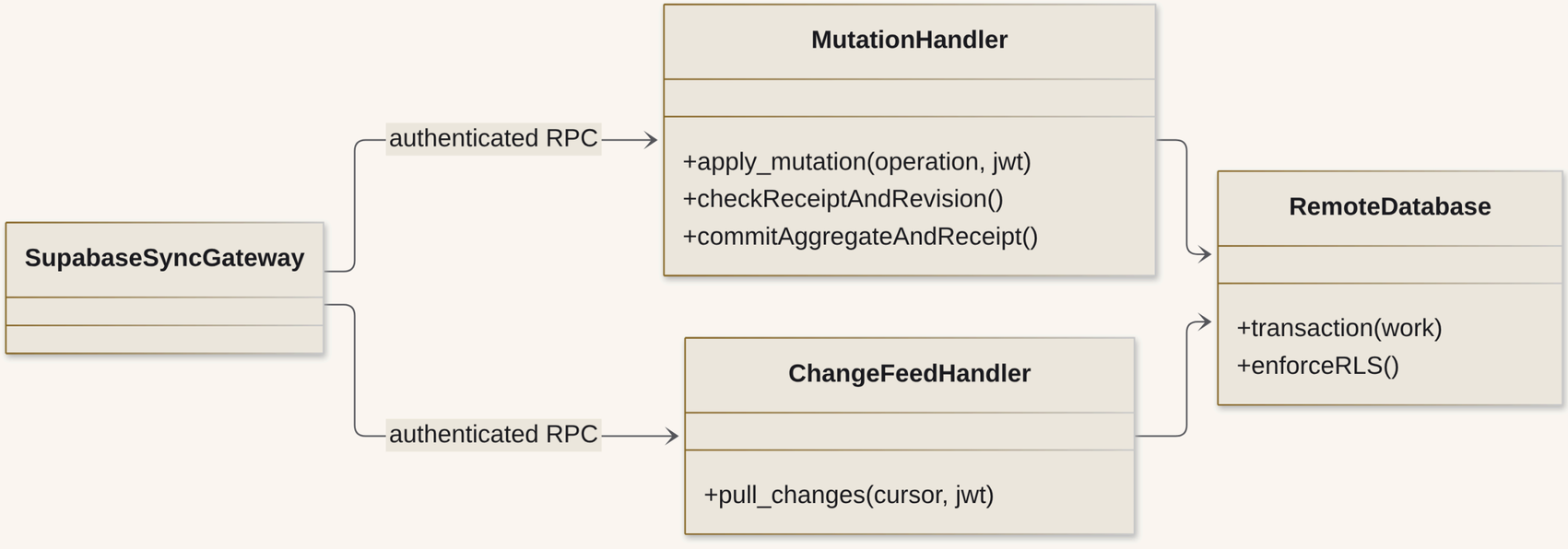
SyncGateway is an interface, so sync is tested without a network.

In the full diagram

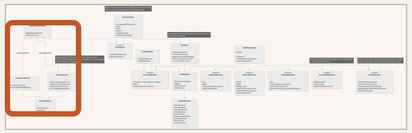


CLASS DIAGRAM · PERSISTENCE AND SYNC · ZOOM 2

Database handlers on Supabase



In the full diagram



MutationHandler: checks receipt and revision, then commits the change and its receipt together.

ChangeFeedHandler: returns each lifter's changes in order.

RemoteDatabase: transactions plus row-level security.

THE ITERATION LOOP

We changed diagrams, never code

1 Revised 1-pagers
professor feedback applied

2 Mermaid diagrams
plus scope and contracts

3 Pull request
reviewed, then merged

4 One-shot generation
one autonomous session

5 Evaluate
simulator, tests, TestFlight

6 Feedback → new IDs
UI-xx, TR-xx, BL-xx

Step 6 loops back to step 2, up to three rounds per model. Every round records the model, design commit, PR, tests, and what is still unverified.

The BL-06 fix for faster set entry lives on a separate phone-release branch, outside the assessed one-shot output.

PROMPTS USED TO TRIGGER GENERATION

One long brief, then short round prompts

Round 1 one-shot brief

- Finish in ONE autonomous session; never wait for a reply
- Read every document and diagram; map each story ID in REQUIREMENTS.md
- Expo + TypeScript, SQLite, Supabase; no placeholders or fake success
- Build, test and run on the iOS simulator; report status truthfully
- Deliver README, REQUIREMENTS, ASSUMPTIONS and DEMO files

Later rounds

OpenAI round 2: “add liquid glass buttons … a more dark, premium aesthetic”

OpenAI round 3: a fresh app from the planned-training design (TR-01–13)

Claude round 2: “too zoomed in … an actual body … male or female … feels laggy”

MODEL A · OPENAI ASTRA 6.1 · HIGH EFFORT · IN CODEX

Three rounds, ending in Formwork

Round 1 · Nov 4 · PR #46

Input: 31 stories, 42 diagrams

Built: typed set logging, previous performance, history, graphs, templates, kg/lb, offline sync

Verified: 7 tests with local Supabase; simulator test saved during an API outage → Uploaded

Round 2 · Nov 8 · PR #49

Input: UI-01–03, a dark graphite and champagne look with Liquid Glass

Verified: iOS build and UI test, 6 tests, contrast 6.95–15.42:1

Skipped: Supabase outage test

Round 3 · Nov 9 · PR #50

Input: 50 diagrams with TR-01–13

Built: Formwork, a fresh app: routines, rest timer and Live Activity, 876-exercise catalog

Feedback: a separate form for every set was too slow (BL-06)

MODEL B · CLAUDE OPUS 5.5 · HIGH EFFORT · IN CLAUDE CODE

Two rounds, building Repwise

Round 1 · Nov 9 · PR #47

Input: the same 50-diagram design (BL-06 already in the backlog)

Built: inline set rows, rest timer, muscle map, routines, reports, CSV

Verified: 41 local + 9 backend tests; hosted Supabase; TestFlight

Round 2 · Nov 10 · PR #48

Requirements first: UI-04 density, UI-05 speed, TR-07A body figure, TR-06A form guide; 5 diagrams updated

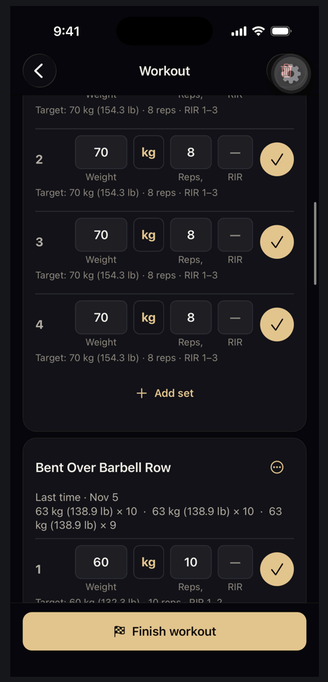
Lag root cause: every set reloaded every tab; now only affected screens reload

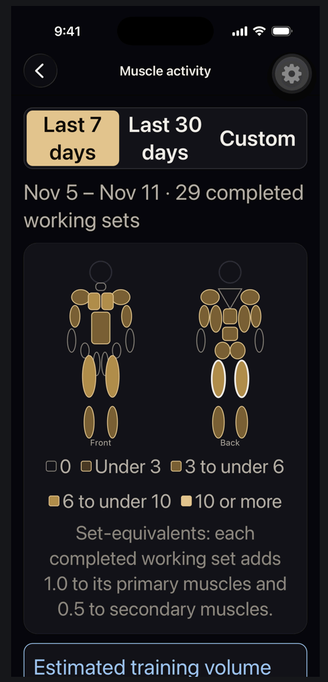
Verified: 44 tests; 9/9 backend

Preserved as git tags claude-one-shot-1 and claude-one-shot-2 · 21

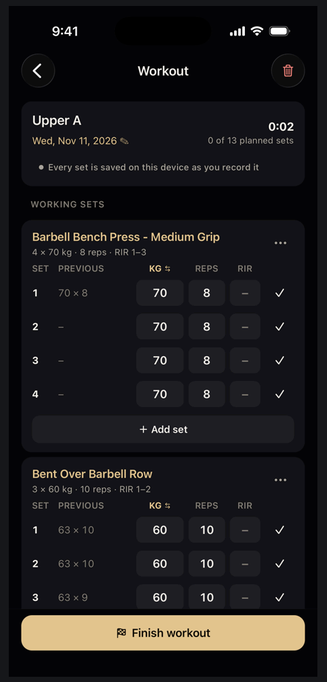
What a diagram change did to the app

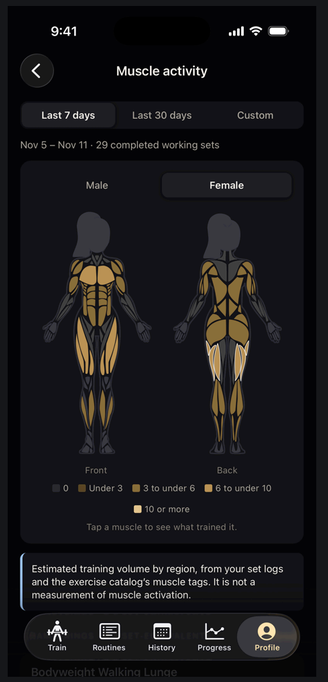
CLAUDE ROUND 1

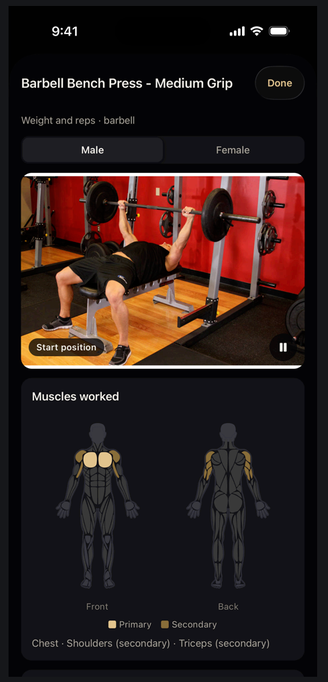




CLAUDE ROUND 2 · UI-04, TR-07A, TR-06A







We changed only the requirements and diagrams. Claude regenerated the code: compact set table, a real body figure, and a How-to-perform guide.

iPhone 17 Pro simulator, iOS 26.5, demo data · 22

Our verdict: Claude’s app was clearly better

	OpenAI · Formwork (round 3)	Claude · Repwise (round 2)
Model	OpenAI Astra 6.1, high effort	Claude Opus 5.5, high effort
Rounds used	3 of 3	2 of 3
Self-reported status	16 verified · 20 partly · 11 unverified (47)	39 verified · 13 not end-to-end (52)
Automated tests	9 unit tests + real-PostgreSQL migration test	44 local + 9 backend tests
Offline sync	Outage → Uploaded on simulator	Offline → upload in backend tests; conflicts shown
Set entry	A form for each set (BL-06)	Inline rows with a checkmark
Deployment	Hosted Supabase · TestFlight in beta	Hosted Supabase · TestFlight (round 1)

After using both apps, we found Claude’s Repwise much better: nicer to use, more polished, and better overall than OpenAI’s Formwork.

LIVE DEMO · ABOUT 4 MINUTES

One script, two apps

OpenAI · Formwork

1. Open demo journal → start Upper body
2. Enter actuals: 11 reps, RIR 2 → Mark done; rest timer
3. Force-close → Resume workout
4. Finish → History → Progress chart drill-down
5. Profile → Muscle activity, monthly report

Claude · Repwise

1. Train → Upper A → start with targets
2. Set row: 7 reps, RIR 2 → ✓; rest timer
3. Force-close → Resume workout
4. Finish → History → Weekly volume
5. How to perform → Muscle map, male/female

Backup if Wi-Fi or TestFlight fails: iPhone simulator with demo data, plus the screenshots in the repo.

PRODUCT BACKLOG FOR PHASE 3

Sprint 1 goal: an offline workout you can trust

ID	Story (user role · stories affected)	Points	Sprint 1
BL-01	Home trainer: confirmed sets survive restart on a real phone (FW-07, FW-09)	5	**Selected**
BL-02	Home trainer: a pending workout uploads once, with conflicts preserved (FW-09)	8	**Selected, split**
BL-04	Experienced lifter: history and graphs reflect corrections (FW-17, 18, 20, 22)	5	If capacity
BL-03	Returning lifter: kg and lb preserved and shown consistently (FW-10, 30, 31)	5	Later
BL-05	Returning lifter: 200% text without missing controls (FW-29)	3	Later
BL-06	Lifter: enter weight and reps on the row and check it off (FW-02, TR-02)	[__]	Device check

Following Sommerville: items written as user-role stories with acceptance checks, sized in story points, and ordered by value and risk. There is no measured velocity yet, so the sprint is an ordered selection, not a promise.

WHAT WE LEARNED

Diagrams set the structure; using the app set the feel

Diagrams as a contract

Both models built the same local-first layers and sync protocol from them

Feel needs a phone

Density, lag and slow set entry only showed up in real use, and became UI-01–05 and BL-06

Evidence over claims

Each round records its model, commit, tests and what is still unverified

Questions?

github.com/SebasLopez2005/SWEProject1